Ensuring Data Integrity with Hash Codes

01/04/2023

A hash value is a numeric value of a fixed length that uniquely identifies data. Hash values represent large amounts of data as much smaller numeric values, so they are used with digital signatures. You can sign a hash value more efficiently than signing the larger value. Hash values are also useful for verifying the integrity of data sent through insecure channels. The hash value of received data can be compared to the hash value of data as it was sent to determine whether the data was altered.

This topic describes how to generate and verify hash codes by using the classes in the System.Security.Cryptography namespace.

Generating a Hash

The hash classes can hash either an array of bytes or a stream object. The following example uses the SHA-256 hash algorithm to create a hash value for a string. The example uses Encoding.UTF8 to convert the string into an array of bytes that are hashed by using the SHA256 class. The hash value is then displayed to the console.

C#

```
using System;
using System.IO;
using System.Security.Cryptography;
using System.Text;

string messageString = "This is the original message!";

//Convert the string into an array of bytes.
byte[] messageBytes = Encoding.UTF8.GetBytes(messageString);

//Create the hash value from the array of bytes.
byte[] hashValue = SHA256.HashData(messageBytes);

//Display the hash value to the console.
Console.WriteLine(Convert.ToHexString(hashValue));
```

This code will display the following string to the console:

Console

```
67A1790DCA55B8803AD024EE28F616A284DF5DD7B8BA5F68B4B252A5E925AF79
```

Verifying a Hash

Data can be compared to a hash value to determine its integrity. Usually, data is hashed at a certain time and the hash value is protected in some way. At a later time, the data can be hashed again and compared to the protected value. If the hash values match, the data has not been altered. If the values do not match, the data has been corrupted. For this system to work, the protected hash must be encrypted or kept secret from all untrusted parties.

The following example compares the previous hash value of a string to a new hash value.

C#

```
using System;
using System.Linq;
using System.Security.Cryptography;
using System.Text;

//This hash value is produced from "This is the original message!"
//using SHA256.
byte[] sentHashValue =
Convert.FromHexString("67A1790DCA55B8803AD024EE28F616A284DF5DD7B8BA5F68B4B252A5E925AF
79");

//This is the string that corresponds to the previous hash value.
string messageString = "This is the original message!";

//Convert the string into an array of bytes.
byte[] messageBytes = Encoding.UTF8.GetBytes(messageString);

//Create the hash value from the array of bytes.
byte[] compareHashValue = SHA256.HashData(messageBytes);

//Compare the values of the two byte arrays.
bool same = sentHashValue.SequenceEqual(compareHashValue);

//Display whether or not the hash values are the same.
if (same)
{
    Console.WriteLine("The hash codes match.");
}
else
{
```

```
    Console.WriteLine("The hash codes do not match.");
}
```

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- ASP.NET Core Data Protection